

04 — Frontend Pages and Routes

Product: Endpoint IQ **Document:** Pages and Routes

Route Table Overview

Route	Page	MVP?
/login	Login	Yes* (see Open Decision)
/dashboard	Dashboard / Projects overview	Yes
/projects	Project List	Yes
/projects/new	Project Creation	Yes
/projects/:projectId	Project Overview	Yes
/projects/:projectId/endpoint	Endpoint Explorer	Yes
/projects/:projectId/roles	Role Configuration	Yes
/projects/:projectId/credentials	Credential Configuration	Yes
/projects/:projectId/test-runs	Test Run List	Yes
/projects/:projectId/test-runs/:runId	Test Run Monitor / Detail	Yes
/projects/:projectId/results	Results Dashboard (latest/selected run)	Yes
/projects/:projectId/reports	Report View / Download	Yes

Open Decision: /login presumes a platform-level authentication page. As noted in 02-Frontend-Architecture.md, the source document does not define platform-user login — only target-API role login (which is a *feature* of the product, not how the developer accesses Endpoint IQ itself). Recommend keeping /login in scope for MVP under the assumption a basic auth layer exists, but confirm with backend before implementation.

/dashboard and /projects may be the same page (a project list *is* a reasonable dashboard for MVP) — see page definition below; this is called out explicitly rather than silently merged.

/login

- **Purpose:** Authenticate the platform user (developer/QA engineer) into Endpoint IQ.
 - **Route:** /login
 - **User actions:** Enter credentials, submit, see validation/auth errors.
 - **Components:** LoginForm, Button, Toast.
 - **Data required:** None on load.
 - **API calls:** POST /auth/login — **Proposed API Contract** (not defined in source; see 05-Frontend-API-Integration.md).
 - **Navigation:** On success → /dashboard.
 - **Loading state:** Button spinner while submitting.
 - **Empty state:** N/A.
 - **Error state:** Inline “invalid credentials” / network error message.
 - **Success state:** Redirect to /dashboard.
-

/dashboard

- **Purpose:** Landing page after login; surfaces the user's projects and recent activity.
- **Route:** /dashboard
- **User actions:** Navigate to a project, start a new project.
- **Components:** ProjectCard list, EmptyState, Button (New Project).
- **Data required:** List of the user's projects (with latest run status per project, if available).
- **API calls:** GET /projects (see 05-...).
- **Navigation:** Click project → /projects/:projectId. Click "New Project" → /projects/new.
- **Loading state:** Card grid skeleton.
- **Empty state:** "Create your first project" prompt.
- **Error state:** Generic retry banner.
- **Success state:** Project cards rendered, each showing name, endpoint count, last run status.

Open Decision: Whether /dashboard is a distinct page from /projects or an alias is not defined in the source. MVP recommendation: /dashboard = /projects (same page, two route aliases) to avoid duplicate implementation; this can be revisited once activity-feed-style features are scoped.

/projects

- **Purpose:** Full list/management view of all projects.
 - **Route:** /projects
 - **User actions:** Search/filter projects, create new, open, delete.
 - **Components:** ProjectTable or ProjectCard grid, SearchInput, Button.
 - **Data required:** All projects for the current user.
 - **API calls:** GET /projects.
 - **Navigation:** → /projects/new, → /projects/:projectId.
 - **Loading/Empty/Error/Success:** Same pattern as /dashboard.
-

/projects/new

- **Purpose:** Create a new Endpoint IQ project (name + Swagger/OpenAPI URL), kicking off the import flow.
 - **Route:** /projects/new
 - **User actions:** Enter project name, enter Swagger URL, submit.
 - **Components:** ProjectForm, Button, Toast.
 - **Data required:** None on load.
 - **API calls:** POST /projects (creates project), then POST /import (triggers spec import — explicitly defined in source §3.1).
 - **Navigation:** On success → /projects/:projectId/endpoints (so the user immediately sees import results).
 - **Loading state:** Submit button spinner; if import is synchronous-feeling but actually async, show an import-in-progress state before redirecting.
 - **Empty state:** N/A.
 - **Error state:** Invalid URL / unreachable spec / parse failure — surfaced per §10 Risks ("Bad OpenAPI Spec... give user clear error message").
 - **Success state:** Redirect with success toast.
-

/projects/:projectId

- **Purpose:** Project overview/hub — summary stats and quick links into each workflow stage.
 - **Route:** /projects/:projectId
 - **User actions:** Navigate to endpoints/roles/credentials/test-runs/results/reports; edit project settings.
 - **Components:** ProjectSummaryCard, WorkflowStepNav, RecentRunsList.
 - **Data required:** Project detail, endpoint count, role count, latest test run summary.
 - **API calls:** GET /projects/:projectId, GET /projects/:projectId/test-runs?limit=5.
 - **Navigation:** Sidebar-driven to all child routes.
 - **Loading state:** Summary card skeleton.
 - **Empty state:** If no endpoints imported yet, prompt to import.
 - **Error state:** Project not found / access denied.
 - **Success state:** Full overview rendered.
-

/projects/:projectId/endpoints

- **Purpose:** View all endpoints discovered from the imported OpenAPI spec.
 - **Route:** /projects/:projectId/endpoints
 - **User actions:** Search/filter by method or path, view endpoint schema detail, re-import spec.
 - **Components:** EndpointTable, MethodBadge, SchemaPreviewModal, SearchInput.
 - **Data required:** List of endpoints (method, path, schemas) per the endpoints table in the data model.
 - **API calls:** GET /projects/:projectId/endpoints.
 - **Navigation:** → Roles (next workflow step).
 - **Loading state:** Table skeleton.
 - **Empty state:** “No endpoints found — import a spec” (link back to import).
 - **Error state:** Load failure banner.
 - **Success state:** Full endpoint table rendered.
-

/projects/:projectId/roles

- **Purpose:** Define the roles the platform will test against (Admin, Client, Company, etc.).
 - **Route:** /projects/:projectId/roles
 - **User actions:** Add/edit/delete a role.
 - **Components:** RoleList, RoleForm, ConfirmDialog.
 - **Data required:** Existing roles for the project.
 - **API calls:** GET /projects/:projectId/roles, POST/PUT/DELETE /projects/:projectId/roles/:roleId
 - **Navigation:** → Credentials (next step).
 - **Loading state:** List skeleton.
 - **Empty state:** “No roles defined — add your first role.”
 - **Error state:** Save/delete failure toast.
 - **Success state:** Updated role list with confirmation toast.
-

/projects/:projectId/credentials

- **Purpose:** Provide login credentials per role so the Auth Service can obtain JWTs for testing.
 - **Route:** /projects/:projectId/credentials
 - **User actions:** Enter/edit credentials for each role; (optionally) verify login.
 - **Components:** CredentialForm (per role), PasswordInput, Button.
 - **Data required:** Role list; existing credential *metadata* (never raw passwords — write-only per security model in §6).
 - **API calls:** GET /projects/:projectId/roles, POST/PUT /projects/:projectId/credentials.
 - **Navigation:** → Test Runs (next step).
 - **Loading state:** Form skeleton.
 - **Empty state:** Prompt per role with no credentials yet.
 - **Error state:** Save failure / invalid credential format.
 - **Success state:** Confirmation that credentials were securely saved (never echo the password back).
-

/projects/:projectId/test-runs

- **Purpose:** List all past and current test runs for the project; entry point to start a new one.
 - **Route:** /projects/:projectId/test-runs
 - **User actions:** Start new run, open an existing run.
 - **Components:** TestRunTable, StatusPill, Button (Start Test Run).
 - **Data required:** List of test_runs (id, start time, status) per data model.
 - **API calls:** GET /projects/:projectId/test-runs, POST /projects/:projectId/test-runs (start run).
 - **Navigation:** → /projects/:projectId/test-runs/:runId.
 - **Loading state:** Table skeleton.
 - **Empty state:** “No test runs yet — start your first run.”
 - **Error state:** Start-run failure (e.g. missing credentials) with actionable message.
 - **Success state:** New run appears with “Running” status; auto-navigate to its detail page.
-

/projects/:projectId/test-runs/:runId

- **Purpose:** Monitor a test run in progress (or review a completed one) — job progress, live/final pass-fail counts.
- **Route:** /projects/:projectId/test-runs/:runId
- **User actions:** Watch progress, cancel run (see Open Decision), navigate to results once complete.
- **Components:** RunProgressBar, RunStatsSummary, StatusPill, LiveJobList (optional).
- **Data required:** Run status, jobs completed/total, per §4.5 Orchestrator’s progress tracking.
- **API calls:** GET /projects/:projectId/test-runs/:runId, plus WebSocket subscription for live updates (REST/WS link in architecture diagram) — see 05-Frontend-API-Integration.md.
- **Navigation:** On completion → /projects/:projectId/results (or stays, with a “View Results” button appearing).
- **Loading state:** Shell renders immediately; progress bar starts at 0/total as data streams in.
- **Empty state:** N/A (a run always has jobs once started).

- **Error state:** Run failed to start / worker errors surfaced at run level (per §10 Risks: target API down, queue overload).
- **Success state:** Progress bar reaches 100%; status transitions to Completed; “View Results” CTA appears.

Open Decision: Cancel-run capability is not explicitly defined in source. Flag as **Proposed API Contract** if desired for MVP.

/projects/:projectId/results

- **Purpose:** Results dashboard for the latest (or a selected) completed test run — pass/fail summary, endpoint result table.
- **Route:** /projects/:projectId/results (optionally ?runId= to view a non-latest run)
- **User actions:** Filter/sort results, open a failed endpoint’s detail, jump to AI analysis, jump to report.
- **Components:** ResultsSummaryStats, EndpointResultTable, StatusPill, Tabs (Overview/Failed/AI Analysis/Report — shared with run detail).
- **Data required:** test_results for the run, joined with endpoints.
- **API calls:** GET /projects/:projectId/test-runs/:runId/results.
- **Navigation:** Row click → failed endpoint detail (modal or nested route, e.g. /projects/:projectId/test-runs/:runId/results/:resultId).
- **Loading state:** Summary + table skeleton.
- **Empty state:** N/A once a run exists (but “no failures” is a valid, positively-framed success state).
- **Error state:** Load failure banner.
- **Success state:** Full results rendered, failures highlighted.

Full component/interaction detail for this page and its children is in **07-Frontend-Test-Results-and-AI-Report-UI.md**.

/projects/:projectId/reports/:reportId

- **Purpose:** View report summary and download the final PDF/HTML file.
 - **Route:** /projects/:projectId/reports/:reportId
 - **User actions:** Review summary, download file.
 - **Components:** ReportSummary, Button (Download).
 - **Data required:** reports record (file_url, summary_text) per data model.
 - **API calls:** GET /projects/:projectId/reports/:reportId, file download via file_url.
 - **Navigation:** Reached from Results page or Test Run detail once report is ready.
 - **Loading state:** Summary skeleton.
 - **Empty state:** “Report not yet generated” if run is incomplete.
 - **Error state:** Report generation failed / file unavailable.
 - **Success state:** Summary shown, download succeeds.
-

Reusable Components Needed Across Pages

Component	Used on
AppLayout (sidebar + header)	All authenticated pages
StatusPill	Results, Test Runs, Endpoints, AI Analysis
EndpointTable / MethodBadge	Endpoints, Results
ConfirmDialog	Roles, Credentials, Projects, Test Runs (cancel)
Toast	All pages (feedback)
Skeleton variants	All data-loading pages
EmptyState / ErrorState	All list/detail pages
JsonViewer / CodeBlock	Endpoints (schema), Results (request/response), AI Analysis
ProgressBar	Test Run detail
AIAnalysisCard	Results, AI Analysis section

Cross-references: 05-Frontend-API-Integration.md (endpoint contracts referenced above), 06-Frontend-State-and-Data-Models.md (data shapes), 07-Frontend-Test-Results-and-AI-Report-UI.md (Results/AI/Report page detail).